

Complexity Analysis of USEI10

It'll be considered:

- N : Total number of stations in the KD-Tree;
- k : Number of results found within the radius;
- s : Average number of stations per node;
- L : Average length of the station name (String).

StationSearchService Class:

StationSearchService() (Constructor): The constructor is responsible for fetching the shared, global data structure.

Code:

```
22: public StationSearchService() {
23:     this.stationTree = StationRepository.getInstance().getKdtree();

25: if (this.stationTree == null) {
26:     throw new IllegalStateException("Critical Error: KDtree not
found in the Repository.");
27: }
28: }
```

- StationRepository.getInstance().getKdtree() call: $O(1)$
- Null check and exception throw: $O(1)$
- Total: $O(1)$
- Initializes the service by fetching the singleton instance of the tree. It runs in constant time as it involves only reference assignments.

FindStationsInRadius(): This method declares the algorithm's complexity by calling the recursive search.

Code:

```
27: public AVLTree<StationDistanceResult> findStationsInRadius(double
targetLat, double targetLon, double radiusKm) {
28:     AVLTree<StationDistanceResult> results = new AVLTree<>();
29:     if (stationTree.getRoot() == null) {
30:         System.out.println("WARNING: The KD-Tree is empty. Make sure you
have run the 'Build KD-Tree' option (USEI07) first.");
31:         return results;
32:     }
34:     recursiveRadiusSearch(stationTree.getRoot(), targetLat, targetLon,
radiusKm, results);
}
```

```

    35: return results;
}

```

- New AVL tree<>() : $O(1)$
- Station.Tree.getRoot() check: $O(1)$
- recursiveRadiusSearch(...) call: $O(\sqrt{N} + k \log k)$
- Total: $O(\sqrt{N} + k \log k)$
- The method complexity is inherited from the recursive call on line 34. This expression is the sum of the spatial search cost ($O(\sqrt{N})$) and the insertion/sorting cost ($O(k \log k)$).

Type: Non-Deterministic

RecursiveRadiusSearch(): This is the core algorithm that combines spatial traversal with ordered insertion.

Code:

```

    38: private void recursiveRadiusSearch(KDnode node, double targetLat,
double targetLon,

    39: double radiusKm, AVLTree<StationDistanceResult> results) {
    40: if (node == null) return;

    42:for (RailwayStation station : node.getStations()) {

    45: double distance = DistanceUtils.haversineDistance(targetLat,
targetLon,
    46:         station.getLatitude(),
    47:         station.getLongitude());

    49: if (distance <= radiusKm) {
    50:     results.insert(new StationDistanceResult(station, distance));
    51: }
    52:}

    54:int axis = node.getAxis();

    ...

    59: double axisDistance;
    60: if (axis == 0) {
    61:     axisDistance = DistanceUtils.haversineDistance(targetLat,
targetLon, node.getLatitude(), targetLon);
    62: } else {
    63:     axisDistance = DistanceUtils.haversineDistance(targetLat,
targetLon, targetLat, node.getLongitude());
    64:}
    66: KDnode nearChild = (targetCoord < nodeCoord) ? node.getLeft() :
node.getRight();
    67:KDnode farChild = (targetCoord < nodeCoord) ? node.getRight() :
node.getLeft();
    69: recursiveRadiusSearch(nearChild, targetLat, targetLon, radiusKm,
results);

    71:if (axisDistance <= radiusKm) {
    72:     recursiveRadiusSearch(farChild, targetLat, targetLon, radiusKm,
results);

```

```
73:}
74:}
```

- Loop over bucket (lines 49-57): Iterates s times;
- haversineDistance: $O(1)$;
- results.insert(...): $O(\log k)$;
- Pruning logic (lines 61, 71): $O(1)$;
- Recursive Calls (lines 69, 72): Visits nodes proportional to $\sqrt{N}$ (in average 2D case).
- Total: $O(\sqrt{N} + k \log k)$
- The bucket loop iterates s times, performing insertions that cost $O(\log k)$. The recursive structure uses pruning ($\text{axisDistance} \leq \text{radiusKm}$) to limit the search to the region boundary, resulting $O(\sqrt{N})$ nodes visited.

GenerateDensitySummary(): Aggregates the search results into summary maps.

Code:

```
76: public Map<String, Map<String,
Integer>>generateDensitySummary(AVLTree<StationDistanceResult> results) {
77:     Map<String, Integer> countryCounts = new HashMap<>();
78:     Map<String, Integer> isCityCounts = new HashMap<>();
81:     for (StationDistanceResult result : results.inOrder()) {
82:         RailwayStation station = result.getStation();
84:         String countryKey = (station.getCountry() != null) ?
station.getCountry().toString() : "UNKNOWN";
85:         countryCounts.put(countryKey,
countryCounts.getDefault(countryKey, 0) + 1);
87:         String cityStatus = station.isCity() ? "City" : "Not City";
88:         isCityCounts.put(cityStatus,
isCityCounts.getDefault(cityStatus, 0) + 1);
89:     }
90: }

92: Map<String, Map<String, Integer>> summaries = new HashMap<>();
93: summaries.put("byCountry", countryCounts);
94: summaries.put("byCityStatus", isCityCounts);
95: return summaries;
96: }
97: }
```

- results.inOrder() traversal: $O(k)$;
- HashMap operations: $O(1)$;
- Total: $O(k)$;
- The method iterates through the already sorted AVL tree containing: k results. Since map operations are constantly on average, the complexity is linear with respect to the number of results found.

StationDistanceResult Class:

CompareTo(): Defines the sorting logic for the AVL Tree.

Code:

```
28: public int compareTo(StationDistanceResult other) {
30:     int distCompare = Double.compare(this.distance, other.distance);
31:     if (distCompare != 0) {
32:         return distCompare;
33:     }
35:     return
other.getStation().getName().compareTo(this.getStation().getName());
36: }
```

- Double.compare: $O(1)$
- String.compareTo: $O(L)$
- Total: $O(1)$
- The method performs a primitive comparison of doubles and a lexicographical comparison of strings. While technically $O(L)$, the name length is negligible and constant relative to the dataset size N , so it is considered $O(1)$.

Global Complexity Analysis Table

Case	Scenario	Complexity	Dominant Factor
Worst Case	$k \approx N$ (Radius covers all)	$O(N \log N)$	Sorting/Insertion of all elements.
Average Case	k is moderate	$O(\sqrt{N} + k \log k)$	Balanced between Search and Sorting.
Best Case	$k \approx 0$ (Empty region)	$O(\sqrt{N})$	Spatial Tree Traversal only.

